

POLITECHNIKA ŚWIĘTOKRZYSKA
Wydział Elektrotechniki, Automatyki i Informatyki

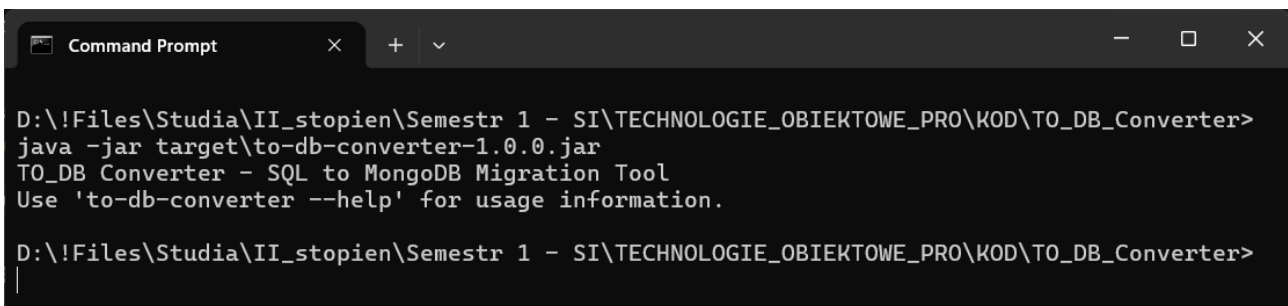
Projekt: Technologie Obiektowe	
Temat: Konwersja danych pomiędzy różnymi modelami.	
Zespół: Marcin Prasalek Kinga Kobiec	Grupa: 1ID21B
	Data oddania: 19.06.2026

1 Wstęp i cel projektu

1.1 Cel projektu

Celem projektu było zaprojektowanie i implementacja uniwersalnego narzędzia migracyjnego **TO_DB Converter**, służącego do automatycznej konwersji i strukturyzacji danych z relacyjnych baz danych (dostępnych przez interfejs **JDBC**) do dokumentowej bazy danych **MongoDB**.

Aplikacja została przygotowana jako w pełni funkcjonalne narzędzie konsolowe (CLI) z interaktywnym, tekstowym kreatorem konfiguracji (TUI), co pozwala na wygodne mapowanie tabel i relacji bezpośrednio w oknie terminala.



```
D:\!Files\Studia\II_stopien\Semestr 1 - SI\TECHNOLOGIE_OBIEKTOWE_PRO\KOD\TO_DB_Converter>
java -jar target\to-db-converter-1.0.0.jar
TO_DB Converter - SQL to MongoDB Migration Tool
Use 'to-db-converter --help' for usage information.

D:\!Files\Studia\II_stopien\Semestr 1 - SI\TECHNOLOGIE_OBIEKTOWE_PRO\KOD\TO_DB_Converter>
```

Rys. 1: Ekran powitalny aplikacji w konsoli po wpisaniu komendy bez parametrów.

1.2 Problem modelowania danych w NoSQL (Zasada access patterns)

Tradycyjne relacyjne bazy danych (takie jak PostgreSQL) opierają się na porządkowaniu danych w postaci normalnej (zazwyczaj 3NF), co eliminuje redundancję (powtarzanie danych), ale zmusza do wykonywania kosztownych operacji **JOIN** przy każdym odczycie.

Przeniesienie tabel do NoSQL (MongoDB) 1:1 jako płaskie kolekcje jest kardynalnym błędem projektowym. MongoDB nie posiada tradycyjnych złączeń **JOIN** realizowanych w locie z wysoką wydajnością. W świecie NoSQL strukturę dokumentu projektuje się pod kątem tego, **jak aplikacja będzie odpytywać bazę (tzw. access patterns / wzorce dostępu)**.

Nasze narzędzie rozwiązuje ten problem. Zamiast bezmyślnie kopiować wiersze, analizuje ono strukturę kluczy obcych (FK) w SQL i pozwala na podjęcie decyzji:

- Czy powiązane dane należy **zagnieździć** (*EMBED* – np. pozycje zamówienia wewnątrz zamówienia), co skraca czas odczytu do jednego zapytania.
- Czy należy pozostawić je jako **referencje** (*REFERENCE* – w osobnych kolekcjach), aby uniknąć przekroczenia twardego limitu wielkości dokumentu w MongoDB (16 MB) i antywzorca nieograniczonych tablic (*unbounded arrays*).

2 Architektura systemu i wzorce obiektowe (GoF)

2.1 Szkielet klas i podział na warstwy (Clean Architecture)

Projekt został zaprojektowany zgodnie z zasadami **Clean Architecture** oraz zasadą pojedynczej odpowiedzialności (**Single Responsibility Principle – SRP**). Warstwa prezentacji (konsola, kreator TUI, parsowanie komend CLI) została całkowicie odizolowana od rdzenia biznesowego (silnika ETL).

Szkielet projektu dzieli się na następujące warstwy:

- `com.todbconverter.cli` – odpowiada za obsługę parametrów wejściowych i routowanie komend (Picocli).
- `com.todbconverter.ui` – warstwa prezentacji (TUI), obsługująca interaktywnego kreatora (JLine 3).
- `com.todbconverter.connection` – infrastruktura połączeń sieciowych (JDBC oraz MongoDB Client).
- `com.todbconverter.config` – zarządza odczytem, walidacją i zapisem konfiguracji (`db-converter.properties`).
- `com.todbconverter.core.extractor` – warstwa pobierania danych i schematu (wykorzystująca uniwersalne metadane JDBC).
- `com.todbconverter.core.transformer` – serce systemu, odpowiadające za sortowanie topologiczne grafu i aplikowanie wzorców optymalizacyjnych NoSQL.
- `com.todbconverter.core.loader` – zapisuje przetworzone dokumenty (BSON) do MongoDB i tworzy indeksy.

Dzięki takiemu podziałowi, silnik migracyjny (`CORE`) nie wie nic o istnieniu interfejsu graficznego ani konsolowego. Można go bez zmian w kodzie uruchomić np. jako bibliotekę w aplikacji webowej.

2.2 Implementacja wzorców GoF w kodzie Javy

Aby w pełni wykorzystać paradygmat programowania obiektowego, w systemie świadomie zaimplementowaliśmy cztery klasyczne wzorce projektowe z katalogu gangu czworga (GoF):

2.2.1 Wzorzec Komendy (Command Pattern)

Wszystkie operacje dostępne dla użytkownika z poziomu konsoli (`run`, `wizard`, `validate`, `report`) zostały wydzielone jako niezależne klasy w pakiecie `com.todbconverter.cli.commands` (np. `RunCommand`, `WizardCommand`).

- Każda z tych klas implementuje interfejs `Callable<Integer>` z biblioteki Picocli.
- Klasa startowa `Main.java` pełni jedynie rolę wywołującego (*invoker*), przekazując sterowanie do odpowiedniej komendy. Dzięki temu dodanie nowej komendy do systemu nie wymaga modyfikacji istniejącego kodu wejściowego.

2.2.2 Wzorzec Metody Szablonowej (Template Method Pattern)

Proces ETL (Extract, Transform, Load) ma zawsze stały, niezmienny szkielet. Wdrożyliśmy go w klasie orkiestrującej `ETLPipeline.java`.

- Metoda `execute()` definiuje niezmienną kolejność kroków algorytmu: nawiązanie połączeń → ekstrakcja schematu → ekstrakcja danych → transformacja do dokumentów → zapis do MongoDB.
- Konkretnie zachowania tych kroków są delegowane do dedykowanych serwisów, ale sam szkielet wykonania potoku jest nienaruszalny i zabezpieczony przed błędami kolejności uruchomienia.

2.2.3 Wzorzec Budowniczego (Builder Pattern)

Podczas analizy bazy danych dynamicznie budujemy obiekty przechowujące metadane tabel (`TableMetadata.java`). Obiekt ten ma wiele cech (nazwa, typ tabeli, kolumny, klucze obce, liczba wierszy).

Zamiast pisać nieczytelne konstruktory z wieloma parametrami, wdrożyliśmy statyczną klasę wewnętrzną `TableMetadata.Builder`.

Pozwala to na płynne i czytelne budowanie obiektów w klasie `JDBCSchemaExtractor`, np.:

```
TableMetadata table = TableMetadata.builder()
    .name(tableName)
    .tableType(tableType)
    .addAllColumns(columns)
    .rowCount(rowCount)
    .build();
```

2.2.4 Wzorzec Strategii (Strategy Pattern)

Wzorzec ten użyliśmy do zarządzania strategiami konwersji (EMBED vs REFERENCE). Rejestr strategii `EdgeStrategyRegistry.java` decyduje w locie, jaki algorytm zastosować dla danej krawędzi relacji. Zamiast pisać skomplikowane i zagnieżdżone warunki `if-else` w głównym transformatorze, logika zachowania dla każdej krawędzi jest dynamicznie pobierana z rejestru na podstawie metadanych połączenia i wykonywana polimorficznie.

3 Instrukcja użytkownika (TUI & CLI)

Aplikacja została zaprojektowana tak, aby jej obsługa była intuicyjna zarówno przy automatycznym uruchomieniu skryptami, jak i przy ręcznym konfigurowaniu.

3.1 Uruchamianie komend z poziomu CLI (Picocli)

Program po skompilowaniu do pliku JAR uruchamiamy w terminalu. Główny moduł CLI (oparty o bibliotekę Picocli) obsługuje cztery podstawowe komendy:

1. **run** (uruchomienie migracji):

```
java -jar to-db-converter-1.0.0.jar run --config=db-converter.properties
```

Wykonuje pełny potok ETL na podstawie wskazanego pliku konfiguracyjnego (lub domyślnego `db-converter.properties` w przypadku braku flagi).

2. **validate** (walidacja połączeń):

```
java -jar to-db-converter-1.0.0.jar validate
```

Służy do szybkiego "spingowania" obu baz (PostgreSQL i MongoDB) przed rozpoczęciem dużej migracji.

3. **report** (generowanie raportu HTML):

```
java -jar to-db-converter-1.0.0.jar report --samples=5 --output=raport.html
```

Pobiera próbki danych ze źródła i celu, po czym generuje czytelny, ostylowany raport wizualny w formacie HTML.

- **Krok 2 (Relationship Strategies):** Tabela wykrytych relacji. Użytkownik porusza się strzałkami góra/dół, a strzałkami lewo/prawo (lub spacją) przełącza strategię (EMBED / REFERENCE) dla wybranej krawędzi relacji. Relacje samoreferencyjne i Many-to-Many są automatycznie zablokowane na opcji REFER.
- **Krok 3 (Design Patterns):** Interaktywna matryca tabel i 5 wzorców projektowych MongoDB. Po najechaniu na komórkę i kliknięciu spacji, włącza się dany wzorzec, a program prosi o wpisanie parametrów w dolnej linii komend.
- **Krok 4 (Summary):** Podsumowanie konfiguracji i zapis do pliku db-converter.properties.

```

Command Prompt - java -jar
Design Patterns - per table
Step 3/4
=====

Patterns:
Attr = Group fields into key-value array
Comp = Pre-calculate COUNT/SUM
Subs = Embed recent N records
Appr = Round numeric values to granularity
Outl = Cap + flag outliers in separate collection

Table
Attr  Comp  Subs  Outl  Appr
-----
activity_logs
▶ departments          x  [ ]
employee_details      x
employee_projects
employees              x    x    x
enrollments                x
projects                x

↑↓ row  ↔ pattern  [Space] toggle  [Enter] confirm

```

Rys. 4: Krok 3 (Design Patterns) w Wizardzie – matryca tabel i wzorców NoSQL z zaznaczonymi krzyżykami 'x' przy aktywnych wzorcach.

4 Analiza scenariuszy migracji

Aby przedstawić, jak nasz transformator i wzorce NoSQL modyfikują dane w locie, przeanalizowaliśmy cztery kluczowe scenariusze konwersji na podstawie naszej bazy testowej.

4.1 Scenariusz 1: Relacja Many-to-Many (Czyste M:N vs M:N z ładunkiem)

W bazie relacyjnej mamy dwie tabele łączące: `employee_projects` (czysta tabela M:N – tylko klucze obce) oraz `enrollments` (tabela z ładunkiem – ma kolumny `grade` i `notes`).

4.1.1 Czyste M:N (np. `employee_projects`)

W PostgreSQL dane są pocięte na 3 tabele. W MongoDB, przy strategii `REFERENCE` dla obu krawędzi, system poprawnie zapisuje relację jako osobną, uproszczoną kolekcję łączącą:

Wynik w MongoDB (kolekcja `employee_project`):

```
{
  "_id": { "$oid": "6a2bc1c02817f184dfca5928" },
  "employee_id": 1,
  "project_id": 1
}
```

Dzięki naszej naprawie błędu złożonego klucza głównego, system nie redukuje tych wierszy do relacji 1:1 i poprawnie przenosi wszystkie połączenia.

4.1.2 M:N z ładunkiem (tabela `enrollments` + `Attribute Pattern`)

W PostgreSQL tabela `enrollments` ma dodatkowe dane (`grade`, `notes`). Konfigurujemy dla niej `Attribute Pattern` w pliku `properties`:

```
pattern.attribute.enrollments=details=notes:Notatki,grade:Ocena,enrolled_at:Data
```

System nie tylko przenosi dane łączące, ale też zwija je w elegancką, elastyczną strukturę atrybutów:

Wynik w MongoDB (kolekcja `enrollment`):

```
{
  "_id": { "$oid": "6a2bc1c02817f184dfca5931" },
  "id": 1,
  "student_id": 1,
  "course_id": 101,
  "details": [
    { "key": "Notatki", "value": ["Bardzo dobry", "Polecam"] },
    { "key": "Ocena", "value": 4.5 },
    { "key": "Data", "value": { "$date": "2026-06-12T06:11:15.362Z" } }
  ]
}
```


Warto zauważyć, że natywna tablica tekstowa z PostgreSQL (`notes TEXT[]`) została poprawnie rzutowana przez refleksję (`PgArray`) na tablicę JSON, a kolumny źródłowe zostały bezpowrotnie usunięte z dokumentu głównego, zapobiegając redundancji.

4.2 Scenariusz 2: Wzorzec Subset vs Pełne zagnieżdżenie (EMBED)

Pracownik Jan Kowalski ma w bazie SQL powiązane logi aktywności (tabela `activity_logs`).

4.2.1 Pełne zagnieżdżenie (EMBED)

Gdybyśmy użyli czystej strategii EMBED, Jan otrzymałby w swoim dokumencie tablicę zawierającą wszystkie swoje logi. Gdyby logów było 100 000, dokument przekroczyłby limit 16 MB.

4.2.2 Wdrożenie Subset Pattern (Podzbioru)

Konfigurujemy dla pracownika wzorzec:

```
pattern.subset.employees=activity_logs=3.
```

Główną relację ustawiliśmy jako REFERENCE (pełna lista logów leży bezpiecznie w osobnej kolekcji `activity_log`), ale w dokumencie pracownika zagnieżdżamy **tylko 3 najnowsze logi**:

Wynik w MongoDB (kolekcja `employee` – dokument Jana):

```
{
  "id": 1,
  "name": "Jan Kowalski",
  "recent_activity_logs": [
    {
      "id": 3,
      "action": "logout",
      "details": { "ip": "192.168.1.1" },
      "created_at": { "$date": "2026-06-12T06:11:15.365Z" }
    },
    {
      "id": 2,
      "action": "edit_file",
      "details": { "file": "config.xml", "lines_changed": 15 },
      "created_at": { "$date": "2026-06-12T06:11:15.365Z" }
    },
    {
      "id": 1,
      "action": "login",
      "details": { "ip": "192.168.1.1", "browser": "Chrome" },
      "created_at": { "$date": "2026-06-12T06:11:15.365Z" }
    }
  ]
}
```

Dzięki poprawce, system automatycznie wykrył kolumnę klucza obcego (`employee_id`) i usunął ją ze struktury zagnieżdżonej, co wyeliminowało zbędne powtórzenia danych.

4.3 Scenariusz 3: Wzorzec Outlier (Wartości odstające)

Wzorzec ten chroni przed puchnięciem dokumentów w przypadku tzw. "anomalii". Konfigurujemy: `pattern.outlier.employees=activity_logs=2` (próg wynosi 2 logi).

4.3.1 Pracownik typowy (Anna Nowak – ma tylko 1 log, czyli poniżej progu)

System zagnieżdża jej logi i ustawia flagę `has_extras` na `false`:

```
{
  "id": 2,
  "name": "Anna Nowak",
  "activity_logs": [
    { "id": 4, "action": "login", "details": { "ip": "10.0.0.1", "browser":
"Firefox" } }
  ],
  "has_extras": false
}
```

4.3.2 Pracownik odstający / Outlier (Jan Kowalski – ma 3 logi, czyli powyżej progu)

System zagnieżdża mu tylko pierwsze 2 logi, a flagę `has_extras` ustawia na `true`. Pozostałe logi Jana nie obciążają jego dokumentu, ale są bezpiecznie zapisane w osobnej kolekcji `activity_log`:

```
{
  "id": 1,
  "name": "Jan Kowalski",
  "activity_logs": [
    { "id": 3, "action": "logout" },
    { "id": 2, "action": "edit_file" }
  ],
  "has_extras": true
}
```

4.4 Wzorce Attribute i Approximation

4.4.1 Wzorzec Attribute (Konsolidacja tabeli `employee_details` 1:1)

Konfiguracja:

`pattern.attribute.employee_details=personal=pesel:PESEL,birth_date:Data,bio:Bio.`

Zamiast płaskich pól, system tworzy jednolitą tablicę klucz-wartość:

```
"employee_details": {
  "id": 1,
  "personal": [
    { "key": "PESEL", "value": "90010112345" },
    { "key": "Data", "value": { "$date": "1990-01-01T00:00:00Z" } },
    { "key": "Bio", "value": "Programista Java z 5-letnim doswiadczeniem" }
  ]
}
```

Dzięki naprawieniu błędu strefy czasowej, data 1990-01-01 zapisała się w MongoDB dokładnie o północy w strefie UTC (1990-01-01T00:00:00Z), eliminując problem przesuwania się daty o jeden dzień wstecz (na 31 grudnia).

4.4.2 Wzorzec Approximation (Zaokrąglanie ocen w `enrollments`)

Konfiguracja: `pattern.approximation.enrollments=grade:1` (zaokrąglaj oceny do pełnych jedności).

- Jan Kowalski miał ocenę 4.5. W wyniku zaokrąglenia algorytmem HALF_UP w MongoDB zapisała się wartość 5.
- Piotr Wiśniewski miał ocenę 3.5. W wyniku zaokrąglenia otrzymał ocenę 4.
- Nasza poprawka uchroniła system przed cichym błędem przepełnienia typu (*Integer Overflow*) dla bardzo dużych sum liczbowych.

5 Podręcznik techniczny (Użytkowanie i Testowanie)

Rozdział ten opisuje sposób uruchomienia, konfiguracji oraz testowania systemu z perspektywy zwykłego użytkownika oraz dewelopera pracującego nad kodem źródłowym.

5.1 Instrukcja dla użytkownika końcowego

Użytkownik końcowy wchodzi w interakcję z programem za pomocą przygotowanego, spakowanego pliku wykonywalnego JAR (`to-db-converter-1.0.0.jar`).

5.1.1 Uruchomienie bazy danych w Dockerze

Przed uruchomieniem migracji należy powołać do życia kontenery ze środowiskiem bazodanowym za pomocą przygotowanego pliku `docker-compose.yml`:

```
docker compose up -d
```

Docker automatycznie pobierze obrazy baz, uruchomi PostgreSQL (port 5433) i MongoDB (port 27018) oraz zaimportuje strukturę i przykładowe dane testowe ze skryptów inicjalizacyjnych.

5.1.2 Konfiguracja i migracja

Kreator TUI: Wpisanie komendy `java -jar to-db-converter-1.0.0.jar wizard` uruchamia interaktywny kreator JLine 3. Użytkownik konfiguruje parametry i zapisuje plik `db-converter.properties`.

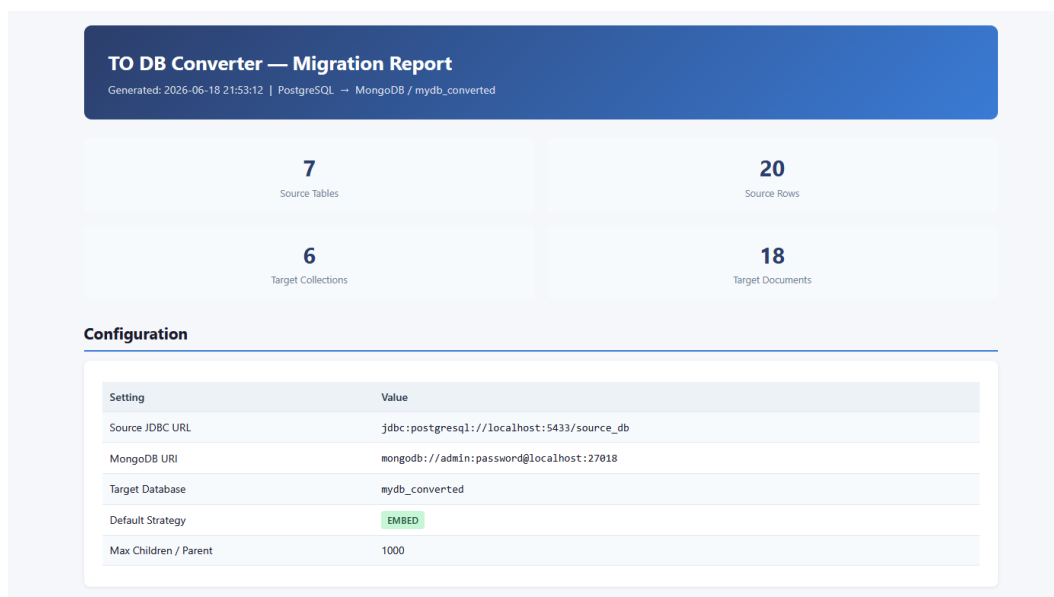
Uruchomienie konwersji: Wpisanie komendy `java -jar to-db-converter-1.0.0.jar run` odpala główny proces ETL. Program na bieżąco raportuje postęp w terminalu.

5.1.3 Generowanie i analiza raportu HTML

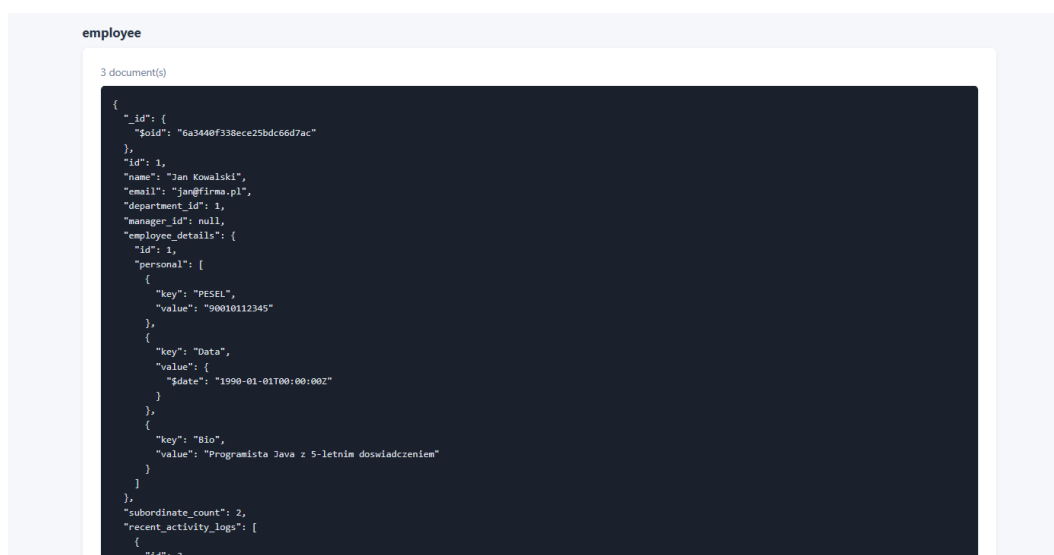
Po udanej migracji użytkownik może wygenerować raport porównawczy komendą:

```
java -jar to-db-converter-1.0.0.jar report
```

Narzędzie wyciągnie próbki danych (`samples`) z PostgreSQL oraz MongoDB, policzy rekordy i wygeneruje samodzielny plik HTML w folderze `reports/report-<timestamp>.html`.



Rys. 5: Wygenerowany raport HTML w przeglądarce - widoczna tabela statystyk.



Rys. 6: Wygenerowany raport HTML w przeglądarce - próbka dokumentów JSON z MongoDB.

5.2 Instrukcja dla programisty (Deweloperska)

5.2.1 Stos technologiczny

- **Język i kompilator:** Java 21 (LTS)
- **Zależności i budowanie:** Maven 3.9+
- **Biblioteki CLI/TUI:** Picocli 4.7.7, JLine 3.27.1, JAnsi 2.4.1
- **Biblioteki NoSQL:** MongoDB Java Driver (Sync Client) 5.6.4, Jackson Databind 2.21.3
- **Biblioteka logowania:** SLF4J + Logback 1.5.15
- **Testowanie:** JUnit 5.11.3, AssertJ 3.26.3, Mockito 5.14.2, H2 Database 2.3.232

5.2.2 Budowanie i kompilacja projektu

Aby zbudować gotowy plik wykonywalny (*shaded JAR*) zawierający wszystkie zależności, należy wywołać w katalogu głównym komendę:

```
mvn clean package
```

Maven utworzy w katalogu `target/` plik `to-db-converter-1.0.0.jar`.

5.2.3 Uruchamianie testów

Aplikacja posiada bogaty pakiet **139 testów** (136 testów jednostkowych i 3 integracyjne). Testy jednostkowe są w pełni izolowane i nie wymagają działających baz.

```
# Uruchomienie tylko testów jednostkowych (szybkich)
mvn test -Dtest="!*IntegrationTest"

# Uruchomienie wszystkich testów, w tym testów integracyjnych z Testcontainers
mvn test
```

Uwaga: Testy integracyjne (`FullMigrationIntegrationTest` oraz `MongoDBLoaderTest`) wykorzystują wbudowaną bazę H2 w pamięci oraz testową bazę MongoDB, aby weryfikować poprawność zapisu danych.

6 Sytuacje skrajne i zabezpieczenia silnika (Safeguards)

Podczas projektowania systemu jednym z kluczowych wyzwań było zabezpieczenie silnika transformacji przed błędami logicznymi, które mogłyby doprowadzić do zawieszenia programu lub cichego uszkodzenia relacji.

6.1 Detekcja i automatyczne rozbijanie cykli oraz samoreferencji

Bazy relacyjne dopuszczają powstawanie zamkniętych cykli zależności (np. Tabela A → B → C → A) oraz samoreferencji (pracownik wskazuje swojego managera w tej samej tabeli). Ślepe zagnieżdżenie (EMBED) takich struktur w NoSQL wywołałoby nieskończoną pętlę i błąd `StackOverflowError`.

Nasze filtry bezpieczeństwa w `UniversalTransformer.detectAndHandleCycles()` rozwiązują to następująco:

- **Samoreferencja:** System przed rozpoczęciem analizy cykli skanuje graf. Jeśli znajdzie krawędź wskazującą na tę samą tabelę (`employees.manager_id`), **bezwzględnie** nadpisuje jej strategię na **REFERENCE** w rejestrze strategii.

- **Cykle:** Silnik uruchamia na grafie algorytm DFS z kolorowaniem wierzchołków. W przypadku wykrycia cyklu, system loguje ostrzeżenie w konsoli, automatycznie **rozbija cykl, wymuszając strategię REFERENCE** na pierwszej krawędzi cyklu i kontynuuje bezpieczne sortowanie topologiczne.

6.2 Bezpiecznik rozmiaru dokumentu (Unbounded Arrays)

Przechowywanie relacji 1:N jako zagnieżdżonej tablicy w dokumencie nadrzędnym (np. logów aktywności w pracowniku) grozi przekroczeniem limitu 16 MB.

- System w metodzie `checkUnboundedArrays()` zlicza rekordy dzieci dla każdej relacji typu `ONE_TO_MANY`.
- Jeśli dla jakiegokolwiek relacji liczba powiązanych dzieci przekracza limit `max_children_per_parent` (domyślnie 1000):
 - System loguje czytelne ostrzeżenie.
 - Automatycznie **degraduje strategię tej relacji do REFERENCE**, wymuszając zapis dzieci w osobnej kolekcji.

6.3 Typy danych (Strefy czasowe i porównania numeryczne)

- **Timezone Shift:** Standardowa konwersja `LocalDate` (np. daty urodzenia) przez strefę systemową JVM potrafiła przesunąć datę o dzień wstecz w MongoDB (z racji strefy UTC). Naprawiliśmy to w `DateUtils.toDate(LocalDate)`, wymuszając strefę `ZoneOffset.UTC`. Data `1990-01-01` zapisuje się w MongoDB dokładnie jako północ w UTC, zapobiegając utracie danych.
- **Integer vs. Long:** Różne sterowniki JDBC zwracają klucze liczbowe jako obiekty `Integer` lub `Long`. Standardowe porównanie `.equals()` w Javie na różnych typach opakowanych zwraca `false` (np. `1.equals(1L) == false`). Doprowadzałoby to do cichego odfiltrowania dzieci i zerowych wyników we wzorcach `Computed` czy `Subset`. Naprawiliśmy to w transformatorze, wprowadzając rzutowanie na `String.valueOf()` w filtrach strumieniowych.

7 Podsumowanie

Projekt z powodzeniem zrealizował cel migracji i transformacji danych z relacyjnego modelu PostgreSQL do zoptymalizowanego modelu dokumentowego MongoDB.

Zastosowanie architektury Clean Architecture pozwoliło na całkowite uniezależnienie silnika biznesowego od konsoli i kreatora TUI. Wykorzystanie obiektowych wzorców projektowych GoF (Command, Strategy, Template Method, Builder) zaowocowało czystym, łatwym w utrzymaniu kodem źródłowym, odpornym na wprowadzanie zmian.

Aplikacja z powodzeniem implementuje **5 zaawansowanych wzorców NoSQL** (Attribute, Computed, Subset, Outlier, Approximation), a jej bezpieczeństwo przed utratą danych, cyklami oraz przepełnieniem dokumentów (limit 16 MB) zostało zagwarantowane przez rozbudowane, wielowarstwowe bezpieczniki logiczne (*safeguards*). Poprawność działania systemu potwierdza pakiet 139 pomyślnie przechodzących testów jednostkowych i integracyjnych.